

Subroutines and parameter passing

- Subroutines and parameter passing
 - Introduction
 - Learning Objectives
 - **Task 1:** Search maximum
 - **Task 2:** I/O functions in assembly
 - Grading

Introduction

In this laboratory you will write and use subroutines (i.e. functions) and examine parameter passing in assembly.

Learning Objectives

- You can implement a subroutine, call it and pass parameters to it.
- You know how to use `push` and `pop`.
- You practice using arrays in assembly.
- You can implement subroutines in assembly to be called in C programs, following the ARM procedure call standard.

Task 1: Search maximum

Write a subroutine to determine the highest value in a table of 32 bit signed integers.

Open the provided program `searchmax`. The file `main.s` implements a test for your subroutine and does **not** require changes. Implement the subroutine `search_max` in the file `search_max.s`.

The address of the array to search is passed in `r0` and the length of the array in `r1`. The return value is stored in `r0`. The subroutine shall return `0x8000_0000` if the arrays length is 0.

Remember to save any of the register `r4..r7` you use to the stack. According to ARM's procedure call standard the registers `r0..r3` do not need to be retained.

Test your implementation by building and flashing the program. On pressing the button T0, the test is started and the result printed to the LCD.

Task 2: I/O functions in assembly

Implement functions to read and write one word (32 bits) in assembly. Call the functions in the main loop to read the DIP switches and write the value to the LEDs. See the function signatures below:

```
1 void write_word(uint32_t address, uint32_t value);
2 uint32_t read_word(uint32_t address);
```

The function `write_word` shall write `value` to `address`. The function `read_word` shall read and return one word from `address`.

Remember to comply with the ARM procedure call standard.

Use the provided, mostly empty, program frame and extend it to implement the described functionality.

Refer to the lecture slides, for an example on how to call an assembly function from C.

Test your implementation on hardware.

Grading

The working programs have to be presented to the lecturer. The student has to understand the solution / source code and has to be able to explain it to the lecturer.

Criteria	Weight
The program meets the requirements of Task 1 .	2/4
The program meets the requirements of Task 2 .	2/4